CLUSTER ENGINEERING · REUSABLE TEMPLATE

Running Training Notebooks on Slurm

How to set up the `slurm/` folder: the eight files that turn a repository of notebooks into a queue-driven pipeline that resumes after a walltime cut, a dead node or a bad seed.

Scope the `slurm/` directory and the thin driver layer it calls
Assumes a shared Slurm cluster, no root, CPU jobs, a home directory on NFS
Independent of algorithm, framework, dataset and environment
Version September 2026

— What this document is

This is a construction guide for one directory. Every file below is a template you copy, rename two variables in, and commit. Nothing here is about a particular model, loss, dataset or environment; the parts that would be project-specific are isolated into a single Python module (`pipeline/stages.py`) that the folder calls but does not contain.

The design assumption throughout: **your experiments live in notebooks, and you want to keep it that way.** The notebook is the thing you can open, read and defend — so the pipeline executes the notebook rather than reimplementing it. The cluster's job is to produce the runs the notebook needs, in parallel, and then to run the notebook once.

Contents

1 The one rule, and the shape it forces

Nothing runs on the login node. Not training, not analysis, not `pip install`. Submitting a job is a control-plane action; everything else is compute and belongs inside an `.sbatch`. Login nodes are shared by everyone on the cluster and are the one resource that has no scheduler protecting it — a stray `pip install torch` there is visible to every other user as a stalled shell.

Once you accept that, the shape of the folder is determined. You need one job to decide *what* to run, an array of jobs to run it, and one job to notice what is left and either try again or move on. That is the whole pipeline:

```
submit_<part>.sh          01_manifest        02_run_chunks           03_analyse
  (login node,      ──>   builds the    ──>  array 0..N-1%K     ──>  merge chunks
   sbatch only)           agreed run          one contiguous           │
                          list for one        slice of the list        ├─ runs left?
                          stage               each, writes one         │    resubmit all three
                                              JSONL file each          │
                                                                       └─ complete?
                                                                            execute the NOTEBOOK,
                                                                            then submit the next
                                                                            stage (or stop)
```

Three properties fall out of this and are worth naming, because every design decision later in the document is in service of one of them.

Property	What it buys you
One agreed list	All array tasks partition the *same* manifest, written once by one job. If each task enumerated the work itself, a result landing between two reads shifts every task's slice: two tasks claim one run, or the last run is claimed by nobody and the stage never finishes.
Append-only ledger	Every finished unit of work appends one self-describing row. Work is redone only if its row is absent. This is what makes a walltime cut, a dead node and a cancelled submission all cost the same thing: the runs that were in flight, and nothing else.
The analysis is the notebook	One implementation of every number you will report, and it is the one you can open. The analyse job executes the `.ipynb`; it does not reimplement it.

Vocabulary used in the templates

Term	Meaning
`STAGE`	One unit of experiment with its own notebook and its own ledger. Stages chain: `E0 → E1 → E2`. If your project is one notebook, you have one stage and the chain is trivial — keep the machinery anyway, it costs nothing.
`GROUPS`	The comma-separated list of sub-populations a stage sweeps: datasets, environments, model sizes, whatever your rows are keyed by. Only appears as an opaque string in the shell.
`N_CHUNKS`	Array width. The manifest is cut into this many contiguous slices.
ledger	The stage's cumulative results table (`artifacts/tables/<STAGE>.csv`), read by the notebook, written only by the merge step.
chunk file	`artifacts/chunks/<STAGE>_chunk_<id>.jsonl` — one file per array task, one JSON object per completed unit of work.

2 Directory layout

```
<project>/
├── submit_part1.sh              ← the only scripts you run by hand
├── submit_part2.sh
├── requirements.txt
│
├── slurm/                       ← THIS DOCUMENT
│   ├── README.md                   why each decision is what it is (no commands)
│   ├── env.sh                      secrets. GITIGNORED. never cat'd
│   ├── _prelude.sh                 sourced by every .sbatch: venv, env vars, guards
│   ├── _submit_lib.sh              sourced by every submit_*.sh: how a stage is submitted
│   ├── 00_create_venv.sbatch       builds the project-owned venv on a compute node
│   ├── 00_verify.sbatch            installs nothing; proves the venv works
│   ├── 01_manifest.sbatch          builds the run list for one stage
│   ├── 02_run_chunks.sbatch        the array
│   ├── 03_analyse.sbatch           merge → requeue, or run the notebook and chain on
│   └── 09_diagnose.sbatch          one process per import, when something native crashes
│
├── pipeline/                    ← the thin driver layer the .sbatch files call
│   ├── stages.py                   THE ONLY PROJECT-SPECIFIC FILE
│   ├── manifest.py
│   ├── chunk.py
│   └── analyse.py
│
├── notebooks/                   ← one per stage; unchanged by any of this
├── artifacts/
│   ├── manifests/  chunks/  tables/  figures/  status/  runs/
└── logs/                        ← gitignored; grows fast
```

TWO CONVENTIONS WORTH ADOPTING VERBATIM

Number the sbatch files by the order you run them. `00_` is setup, `01–03` is the pipeline, `09_` is diagnostics. Someone opening the folder cold — including you in four months — can read the execution order off `ls`.

Keep every command in exactly one place. `slurm/README.md` explains *why* and contains nothing to copy; a top-level `RUNBOOK.md` contains everything you type. A command that appears in two documents will be wrong in one of them within a month.

What does *not* go in `slurm/`

- **No experiment logic.** If an `.sbatch` file knows what a hyperparameter is, that knowledge is in the wrong file. The sbatch scripts know: which venv, which resources, which Python entry point.
- **No secrets in tracked files.** `env.sh` holds the API keys and is in `.gitignore` from the first commit, not from the commit after you notice.
- **No output.** Logs go to `logs/`, results to `artifacts/`.

3 The shared pieces

Three files hold everything the others would otherwise duplicate. Build these first; the four job scripts become short once they exist.

3.1 `env.sh` — the one file that cannot be committed

Sourced by the prelude *if present*, ignored if not. Nothing may depend on it existing, or a fresh clone will not run.

slurm/env.sh

```
#!/bin/bash
# Optional. Sourced by _prelude.sh if present, ignored if not.
#
# The ONLY thing that belongs here is a credential, because it is the one setting that
# cannot live in the repository. Paths, thread counts, backends -- all of those are set in
# _prelude.sh so that a job never depends on this file existing.
#
# GITIGNORED. Never `cat` this file into a log or a chat window.

# export WANDB_API_KEY="..."          # or MLFLOW_TRACKING_TOKEN, HF_TOKEN, ...

export EXPERIMENT_PROJECT="my-project"
export WANDB_SILENT="true"            # compute nodes have no interactive terminal
export WANDB__SERVICE_WAIT="300"      # a slow upload must not hold a training job open
```

Add `slurm/env.sh` to `.gitignore` in the same commit that creates the folder. If you already have a login on the cluster, check whether you need the file at all — most tracking tools write a token to `~/.netrc` or `~/.config/`, and compute nodes share your home directory:

```
grep -s api.wandb.ai ~/.netrc     # already logged in? then env.sh needs no key
```

3.2 `_prelude.sh` — sourced by every job script

Every `.sbatch` needs the same eight lines: change to the project root, activate the venv, source the optional secrets, set the numeric-library thread counts, force a headless plotting backend, and re-check that the variables the submitter exported arrived intact. Writing those eight lines into four files means fixing a bug in four places and forgetting one. Put them here.

slurm/_prelude.sh

```
#!/bin/bash
# Sourced at the top of every .sbatch. Never executed directly.
#
# Everything that must be true before any job of this project does work. One copy, so a
# fix lands everywhere at once.

set -euo pipefail

# ---- 1. project root ------------------------------------------------------------
# Resolved from this file's own location, so the scripts work from any cwd and the path
# appears exactly once in the repository.
PROJECT_ROOT="$(cd "$(dirname "${BASH_SOURCE[0]}")/.." && pwd)"
cd "$PROJECT_ROOT"

mkdir -p logs artifacts/{manifests,chunks,tables,figures,status,runs}

# ---- 2. the project's OWN virtualenv --------------------------------------------
# Not a shared or lab-wide venv. See section 4.1 for why this is not negotiable.
# Override with PROJ_VENV=/some/path if you keep it elsewhere.
VENV="${PROJ_VENV:-$HOME/.venvs/myproject}"
if [ ! -f "$VENV/bin/activate" ]; then
    echo "no venv at $VENV -- run: sbatch slurm/00_create_venv.sbatch" >&2
    exit 1
fi
# shellcheck disable=SC1091
source "$VENV/bin/activate"

# ---- 3. optional secrets --------------------------------------------------------
[ -f slurm/env.sh ] && source slurm/env.sh || true

# ---- 4. a compute node is not a workstation -------------------------------------
export CUDA_VISIBLE_DEVICES=""                # CPU project: no accidental CUDA init path
export OMP_NUM_THREADS="${SLURM_CPUS_PER_TASK:-4}"
export MKL_NUM_THREADS="${SLURM_CPUS_PER_TASK:-4}"
export MPLBACKEND=Agg                         # no display; matplotlib must not look for one
export PYTHONUNBUFFERED=1                     # a killed job must not lose its last 4 KB
export TOKENIZERS_PARALLELISM=false           # (drop if you have no tokenizers)

# Thread counts matter more than they look. The numeric libraries default to "one thread
# per core the NODE has", not per core SLURM gave you. On a 128-core node a 4-cpu task
# spawns 128 threads that fight over 4 cores, and everything runs several times slower
# while looking busy.

# ---- 5. did the submitted variables survive the trip? ---------------------------
# See section 9.1. GROUPS contains commas; N_GROUPS is a plain integer that cannot be
# mangled the same way. If they disagree, the values were damaged in transit and the job
# must refuse rather than silently run a fraction of the experiment.
GROUPS="${GROUPS:-}"
if [ -n "$GROUPS" ]; then
    N_GOT=$(awk -F, '{print NF}' <<< "$GROUPS")
    echo "STAGE=${STAGE:-<unset>}  N_CHUNKS=${N_CHUNKS:-<unset>}"
    echo "GROUPS (${N_GOT}): ${GROUPS}"
    if [ -n "${N_GROUPS:-}" ] && [ "$N_GOT" -ne "$N_GROUPS" ]; then
        echo >&2 "REFUSING: submitter meant ${N_GROUPS} groups; this job received ${N_GOT}."
        echo >&2 "GROUPS was mangled in transit -- almost certainly an"
        echo >&2 "  --export=ALL,GROUPS=a,b,c  call. Submit via slurm/_submit_lib.sh."
        exit 1
    fi
fi

echo "host=$(hostname)  cpus=${SLURM_CPUS_PER_TASK:-?}  job=${SLURM_JOB_ID:-?}"
echo "start $(date -Is)"
```

3.3 `_submit_lib.sh` — the one place that knows how a stage is submitted

All submit scripts source this. Having them share it is what stops three parts of the project from drifting into submitting the same pipeline three slightly different ways.

slurm/_submit_lib.sh

```
#!/bin/bash
# Sourced by submit_*.sh at the repo root. Never executed directly.

default_groups() { echo "alpha,beta,gamma"; }

# Slurm accounts are capped: N running and M submitted at once. Check yours with
#     sacctmgr show assoc user=$USER format=account,maxjobs,maxsubmit
# and set these to match. The manifest and analyse jobs occupy two of the submit slots.
MAX_SUBMIT=32
CONCURRENCY=16            # the "%K" throttle on the array

# Parse and validate IN THE CALLER'S SHELL -- not in a $(...) helper.
#
# A helper called as N=$(parse "$@") runs in a subshell, so its `exit 1` on a bad value
# kills only the subshell and the caller carries on and submits anyway. A guard that does
# not guard is worse than no guard. Setting a variable in the current shell is the fix.
parse_chunks() {
    N_CHUNKS=16
    for arg in "$@"; do
        if [[ "$arg" =~ ^[0-9]+$ ]]; then
            N_CHUNKS="$arg"
        else
            echo "Unrecognized argument: $arg (expected a chunk count)" >&2
            return 1
        fi
    done
    [ "$N_CHUNKS" -ge 1 ] || { echo "N_CHUNKS must be >= 1" >&2; return 1; }
    if [ "$((N_CHUNKS + 2))" -gt "$MAX_SUBMIT" ]; then
        echo "Refusing: N_CHUNKS=$N_CHUNKS plus the manifest and analyse jobs exceeds the" \
             "${MAX_SUBMIT}-job submission cap." >&2
        return 1
    fi
}

submit_stage() {
    local stage="$1" groups="$2" n_chunks="$3" part_end="$4"
    local last=$((n_chunks - 1))

    # THE VARIABLES ARE EXPORTED HERE, NOT LISTED INSIDE --export=...
    #
    # sbatch's --export takes a COMMA-SEPARATED list of NAME=VALUE pairs and offers no way
    # to escape a comma inside a value, so
    #     --export=ALL,STAGE=E0,GROUPS=alpha,beta,gamma,N_CHUNKS=16
    # is read as SIX items: ALL | STAGE=E0 | GROUPS=alpha | beta | gamma | N_CHUNKS=16.
    # `beta` and `gamma` are taken as names of variables to propagate; they do not exist,
    # so they vanish without a warning, GROUPS arrives as "alpha", and the stage runs one
    # group out of three and exits 0. See section 9.1.
    #
    # --export=ALL on its own propagates this shell's whole environment, commas intact.
    export STAGE="$stage" GROUPS="$groups" N_CHUNKS="$n_chunks" PART_END="$part_end" PASS_NO=0
    N_GROUPS=$(awk -F, '{print NF}' <<< "$groups"); export N_GROUPS

    echo "stage=${stage}  chunks=${n_chunks}  groups(${N_GROUPS})=${groups}"

    # A stage that does no parallel work (pure analysis of an earlier stage's outputs) gets
    # the analyse job alone. An empty 16-task array would burn half the submission cap on
    # jobs with nothing to do.
    if grep -q "\"$stage\"" <<< "${ANALYSIS_ONLY_STAGES:-}"; then
        local ajob; ajob=$(sbatch --parsable --export=ALL slurm/03_analyse.sbatch)
        echo "  ${stage}: analyse only -> job ${ajob}"; return 0
    fi

    echo "1/3 manifest..."
    local mjob; mjob=$(sbatch --parsable --export=ALL slurm/01_manifest.sbatch)
    echo "    -> job $mjob"

    echo "2/3 ${n_chunks}-task array (0-${last}%${CONCURRENCY}, after manifest)..."
    local rjob; rjob=$(sbatch --parsable \
        --dependency=afterok:${mjob} \
```

```
        --array=0-${last}%${CONCURRENCY} \
        --export=ALL \
        slurm/02_run_chunks.sbatch)
    echo "    -> job $rjob"

    # afterany, NOT afterok. A task that hit the walltime or died on one bad unit of work
    # still left good rows behind, and noticing what is left is this job's entire purpose.
    echo "3/3 analyse (after every task finishes, however it finishes)..."
    local ajob; ajob=$(sbatch --parsable \
        --dependency=afterany:${rjob} \
        --export=ALL \
        slurm/03_analyse.sbatch)
    echo "    -> job $ajob"

    echo
    echo "Submitted ${stage}: manifest=${mjob} runs=${rjob} analyse=${ajob}"
    echo "Track:  squeue -u \$USER"
    echo
    echo "READ THE MANIFEST LOG FIRST -- it prints how many groups it actually saw:"
    echo "    cat logs/manifest_${mjob}.out"
    echo "It must say ${N_GROUPS}."
}
```

4 The environment jobs

4.1 `00_create_venv.sbatch`

Build the environment *on a compute node*, into a path this project owns alone. Both halves of that sentence matter.

On a compute node, because a large install is exactly the kind of load a login node cannot absorb, and because the node that installs the wheels should be the same *kind* of node that imports them — a wheel compiled for instructions the compute nodes lack fails at import with `SIGILL`, and you want to find that out now.

Owned alone, because a shared lab venv is a shared mutable dependency. Installing your pins into it will move a transitive dependency underneath somebody else's project, and theirs will move one underneath yours. A per-project venv costs a gigabyte and removes an entire category of failure that is very expensive to diagnose.

`slurm/00_create_venv.sbatch`

```
#!/bin/bash
#SBATCH --job-name=proj_venv
#SBATCH --output=logs/venv_%j.out
#SBATCH --error=logs/venv_%j.err
#SBATCH --time=00:50:00
#SBATCH --cpus-per-task=4
#SBATCH --mem=8G
# #SBATCH --partition=<your_partition>

#   sbatch slurm/00_create_venv.sbatch
#   tail -f logs/venv_<jobid>.out

# NOT `set -e`: a failed check here is data to report, not a reason to abort the report.
set -uo pipefail
cd "$(dirname "$0")/.."
mkdir -p logs

VENV="${PROJ_VENV:-$HOME/.venvs/myproject}"
PYBIN="${PROJ_PYTHON:-python3}"

echo "==================== node ===================="
echo "host   : $(hostname)"
echo "os     : $(. /etc/os-release 2>/dev/null && echo "$PRETTY_NAME" || uname -sr)"
echo "arch   : $(uname -m)"
echo "glibc  : $(ldd --version | head -1)"
echo "python : $($PYBIN -V 2>&1)   ($(command -v $PYBIN))"
echo "target : $VENV"

echo; echo "==================== 1. create ===================="
# --clear so a rerun after a failed attempt starts from nothing rather than layering onto
# a half-built environment, which is one good way to produce a broken one.
"$PYBIN" -m venv --clear "$VENV"
source "$VENV/bin/activate"
python -m pip install --upgrade pip setuptools wheel 2>&1 | tail -2

echo; echo "==================== 2. the big framework, FIRST and ALONE ================"
# Pinned to the CPU index deliberately. If torch is instead resolved as a side effect of
# some other requirement, pip takes the default (CUDA) wheel and drags in ~3 GB of nvidia-*
# packages this project cannot use -- plus a CUDA initialisation path that can fail on a
# node with no driver.
pip install --index-url https://download.pytorch.org/whl/cpu torch 2>&1 | tail -3

echo; echo "==================== 3. everything else ===================="
pip install -r requirements.txt 2>&1 | tail -4

echo; echo "==================== 4. verify, ONE PROCESS PER IMPORT ===================="
# A SIGSEGV or a dynamic-linker assertion kills the interpreter outright. No `except` can
# catch either, because neither is an exception. If every import shares one `python -c`,
# the first native crash ends the process and the log says "Segmentation fault" without
# naming which package caused it. One process each means the crash names the culprit and
# the loop continues to the next one.
export PYTHONUNBUFFERED=1
FAILED=""
check() {
    local label="$1" code="$2" rc out
    printf "  %-22s " "$label"
    out=$(mktemp); ( python -u -c "$code" ) >"$out" 2>&1; rc=$?
    tail -2 "$out"; rm -f "$out"
    case $rc in
        0)   return 0 ;;
        132) echo "      ^^ SIGILL: wheel wants CPU instructions this node lacks" ;;
        134) echo "      ^^ SIGABRT: a native library aborted" ;;
        137) echo "      ^^ SIGKILL: out of memory" ;;
        139) echo "      ^^ SIGSEGV: native crash" ;;
        *)   echo "      ^^ exit $rc" ;;
    esac
    FAILED="$FAILED $label"; return 1
}
```

```
check numpy      'import numpy;  print(numpy.__version__)'
check pandas     'import pandas; print(pandas.__version__)'
check pyarrow    'import pyarrow; print(pyarrow.__version__)'
check torch      'import torch;  print(torch.__version__, torch.get_num_threads())'
check matplotlib 'import matplotlib; matplotlib.use("Agg"); print(matplotlib.__version__)'
# ... one line per top-level dependency of the project ...

echo
if [ -n "$FAILED" ]; then
    echo "FAILED:$FAILED"
    echo "If several native packages fail identically, the venv itself is damaged --"
    echo "rerun this script (it uses --clear) before investigating anything else."
    exit 1
fi
echo "venv OK: $VENV"
```

4.2 `00_verify.sbatch` — installs nothing, proves everything

A separate job whose only purpose is to answer "can this cluster run this project?" without spending a queue slot on real work. Run it after the venv build, after any dependency change, and any time a stage fails in a way you cannot explain.

It should do, in order: import every top-level dependency; import every module of your own repository (a typo in an import statement discovered by the array at 3 a.m. is an expensive way to find a typo); construct whatever your project's smallest real object is; and execute *one* genuine training step. That last item is the point — an import check passes on a machine where the first backward pass segfaults.

slurm/00_verify.sbatch

```
#!/bin/bash
#SBATCH --job-name=proj_verify
#SBATCH --output=logs/verify_%j.out
#SBATCH --error=logs/verify_%j.err
#SBATCH --time=00:20:00
#SBATCH --cpus-per-task=4
#SBATCH --mem=8G

source "$(dirname "$0")/_prelude.sh"

echo "--- 1. repository modules import ---"
python - <<'PY'
import importlib, pkgutil, sys
bad = []
for pkg in ("pipeline", "core"):            # your top-level packages
    try:
        m = importlib.import_module(pkg)
    except Exception as e:
        bad.append((pkg, e)); continue
    for mod in pkgutil.walk_packages(m.__path__, pkg + "."):
        try: importlib.import_module(mod.name)
        except Exception as e: bad.append((mod.name, e))
print("FAILED:", bad) if bad else print("all modules import")
sys.exit(1 if bad else 0)
PY

echo "--- 2. one real training step ---"
# Smallest end-to-end unit of work the project has: build it, step it once, assert the
# loss is finite. Substitute your own entry point; keep it under a minute.
python -m pipeline.smoke --steps 1

echo "verify OK"
```

4.3 `09_diagnose.sbatch` — for when something crashes natively

Kept separate and numbered out of the way because you will need it rarely and urgently. It answers three questions the normal logs cannot: which import crashes, what the node actually is, and whether the crash is reproducible on a different node family.

slurm/09_diagnose.sbatch

```
#!/bin/bash
#SBATCH --job-name=proj_diag
#SBATCH --output=logs/diag_%j.out
#SBATCH --error=logs/diag_%j.err
#SBATCH --time=00:20:00
#SBATCH --cpus-per-task=4
#SBATCH --mem=8G
# Pin a node family to test the "is it this hardware?" hypothesis:
# #SBATCH --constraint=<feature>          (see: sinfo -o "%20N %10c %25f")

set -uo pipefail                    # not -e: collect every result, then report
cd "$(dirname "$0")/.."
source "${PROJ_VENV:-$HOME/.venvs/myproject}/bin/activate"

echo "host=$(hostname)  arch=$(uname -m)  kernel=$(uname -r)"
echo "cpu flags: $(grep -m1 ^flags /proc/cpuinfo | tr ' ' '\n' | \
                   grep -E '^(avx|avx2|avx512f|sse4_2)$' | tr '\n' ' ')"
echo "glibc: $(ldd --version | head -1)"
echo "venv : $(which python)"
echo

# One process per import: a SIGSEGV kills only its own probe, so the log names the culprit.
for m in numpy pandas scipy pyarrow matplotlib torch; do
    printf "%-14s " "$m"
    python -u -c "import $m; print('ok', getattr($m,'__version__','?'))" 2>&1 | tail -1
    echo "               exit=$?"
done

echo
echo "--- library resolution for the module that crashes ---"
python -c "import numpy, os; print(os.path.dirname(numpy.__file__))"
# ldd on the offending .so tells you whether it is a missing system library or a damaged one:
#   ldd $(python -c "import numpy,glob,os;
#         print(glob.glob(os.path.dirname(numpy.__file__)+'/core/_multiarray_umath*.so')[0])")
```

READING A NATIVE CRASH

Signature	Means	Do
`ld.so ... Assertion failed`	the `.so` files are damaged	rebuild the venv with `--clear`
`SIGILL` (exit 132)	the wheel needs CPU instructions this node lacks	pin newer nodes with `--constraint` , or install a wheel with a lower baseline
`SIGSEGV` (exit 139)	native crash, cause open	`09_diagnose` : one process per import names it
`SIGKILL` (exit 137)	out of memory, or walltime	`sacct -j <id> --format=MaxRSS,Elapsed,State`

5 The pipeline jobs

These three are short, because the prelude did the setup and the Python did the thinking. That is the intended proportion: an `.sbatch` file is a resource request and one command.

5.1 `01_manifest.sbatch`

slurm/01_manifest.sbatch

```
#!/bin/bash
#SBATCH --job-name=proj_manifest
#SBATCH --output=logs/manifest_%j.out
#SBATCH --error=logs/manifest_%j.err
#SBATCH --time=00:10:00
#SBATCH --cpus-per-task=2
#SBATCH --mem=4G

# Writes artifacts/manifests/<STAGE>.json: the flat, ordered list of every unit of work
# this stage still needs, minus anything already in its ledger. Pure bookkeeping -- and
# still run on a compute node, because nothing runs on the login node.
#
# Its own job, rather than logic inside each array task, so that all N tasks partition ONE
# agreed list. See section 1.
#
# Submitted by slurm/_submit_lib.sh, which exports the variables and passes --export=ALL
# alone. Never hand-write `sbatch --export=ALL,GROUPS=a,b,c` -- sbatch splits that on its
# own commas and delivers GROUPS=a.

source "$(dirname "$0")/_prelude.sh"

python pipeline/manifest.py \
    --stage    "${STAGE}" \
    --groups   "${GROUPS}" \
    --n_chunks "${N_CHUNKS:-16}"
```

5.2 `02_run_chunks.sbatch` — the array

Two numbers in this file deserve thought rather than a default.

The walltime. Look up your cluster's priority bands before choosing it (`scontrol show partition`, and your site's documentation). Many clusters give a substantial priority bonus to jobs under a threshold — asking for six hours when five would do can put you behind everyone who asked for five. Because the analyse job requeues whatever is unfinished, the walltime limits how long one task holds a node, *not* how much work the stage can do. Choose the best priority band, not the biggest number.

The `%K` throttle. `--array=0-31%16` submits 32 tasks and runs at most 16 at once. Set K to your account's concurrent-job cap, not above it: exceeding it does not get you more nodes, it gets your submissions rejected or your priority reduced.

slurm/02_run_chunks.sbatch

```
#!/bin/bash
#SBATCH --job-name=proj_run
#SBATCH --output=logs/run_%A_%a.out
#SBATCH --error=logs/run_%A_%a.err
#SBATCH --time=05:00:00
#SBATCH --cpus-per-task=4
#SBATCH --mem=8G
#SBATCH --array=0-15%16
# #SBATCH --partition=<your_partition>

# Runs one contiguous slice of artifacts/manifests/<STAGE>.json.
#
# SCALING: to change parallelism, edit BOTH the --array line above and the N_CHUNKS the
# submit script exports, to the same number. Keep (N_CHUNKS + 2) within your submit cap.
# The --array line here is overridden by _submit_lib.sh's --array flag; it is kept in the
# file so that a bare `sbatch slurm/02_run_chunks.sbatch` still does something sane.

source "$(dirname "$0")/_prelude.sh"

# ---- TELEMETRY OFF FOR ARRAY TASKS ------------------------------------------------------
# Experiment trackers start a sidecar process per run and reach it over a Unix socket in
# /tmp. Sixteen array tasks start within the same second, so sixteen sidecars race to come
# up against the same NFS home. When that race is lost, every run dies before its first
# step -- and, because the runs recorded nothing, the analyse job sees the same work
# pending and requeues, over and over. This is section 9.2, and it is worth a stage.
#
# Nothing of value is lost: every metric is written to a parquet/CSV file beside the
# checkpoint, and the ANALYSE job -- one process, not sixteen -- logs the stage summary.
# What you give up is a live per-run curve during training.
# Use MODE=offline plus a post-hoc `sync` if you want those curves back.
export WANDB_MODE=disabled

echo "stage=${STAGE} chunk=${SLURM_ARRAY_TASK_ID}/${N_CHUNKS:-16}"

# --minutes against the wall, with a margin: 280 of 300 minutes here. The margin must be
# sized from the SLOWEST unit of work in the stage, not the average -- its purpose is that
# a task never starts something it cannot finish and record. A unit killed mid-way leaves
# no row, so it is simply redone; but it also wasted the whole slot it was killed in.
python pipeline/chunk.py \
    --stage      "${STAGE}" \
    --chunk_id   "${SLURM_ARRAY_TASK_ID}" \
    --n_chunks   "${N_CHUNKS:-16}" \
    --minutes 280 --margin_min 25

echo "end $(date -Is)"
```

5.3 `03_analyse.sbatch` — merge, then requeue or move on

This is the job that makes the pipeline self-healing, and it is the only one that runs the notebook. Give it more memory than the array tasks: it holds the whole ledger and builds every figure.

slurm/03_analyse.sbatch

```
#!/bin/bash
#SBATCH --job-name=proj_analyse
#SBATCH --output=logs/analyse_%j.out
#SBATCH --error=logs/analyse_%j.err
#SBATCH --time=02:00:00
#SBATCH --cpus-per-task=4
#SBATCH --mem=16G

# Merges the chunk files into the stage ledger, then does one of two things:
#
#   work remains   -> resubmit manifest + array + this job, and exit. That is what makes
#                     the pipeline survive a walltime cut, a dead node or a bad seed: the
#                     ledger only grows, so nothing is redone and nothing is lost.
#   stage complete -> execute the stage's NOTEBOOK, which produces the tables, figures and
#                     summary, then submit the next stage if this part has one.
#
# To rerun this job alone (after fixing the notebook, say):
#   export STAGE=E0 GROUPS="alpha,beta,gamma" N_GROUPS=3 N_CHUNKS=16 PART_END=E0 PASS_NO=0
#   sbatch --export=ALL slurm/03_analyse.sbatch
# NOT `sbatch --export=ALL,GROUPS=a,b,c ...` -- sbatch splits that on its own commas.

source "$(dirname "$0")/_prelude.sh"

python pipeline/analyse.py \
    --stage    "${STAGE}" \
    --groups   "${GROUPS}" \
    --n_chunks "${N_CHUNKS:-16}" \
    --part_end "${PART_END:-}" \
    --pass_no  "${PASS_NO:-0}"
```

6 The submit scripts

These live at the repository root, not in `slurm/`, because they are the interface: the only files a person runs by hand. Each is four lines. Their whole job is to name a stage, a group list and where the chain stops.

submit_part1.sh

```
#!/bin/bash
# Part 1: STAGE_A -> STAGE_B, then stop.
#
#   ./submit_part1.sh          # 16 chunks (default)
#   ./submit_part1.sh 8        # 8 chunks
#
# Submits and exits. Nothing computes here -- this runs on the login node.
set -euo pipefail
cd "$(dirname "$0")"
source slurm/_submit_lib.sh

parse_chunks "$@" || exit 1
submit_stage "STAGE_A" "$(default_groups)" "$N_CHUNKS" "STAGE_B"
```

`PART_END` (the last argument) is the safety catch. The analyse job chains to the next stage automatically; `PART_END` is where it stops. Without it, one submission at 6 p.m. can walk the entire experiment graph overnight — which is occasionally what you want and usually not, because the point of stages is that you look at each one's output before paying for the next.

7 The driver layer

Four Python files. Three of them are generic and can be copied between projects unchanged; only `stages.py` knows anything about your experiment.

7.1 `stages.py` — the project-specific boundary

Everything the pipeline needs to know about your experiment, and nothing else. Keeping this interface small is what makes the rest reusable.

pipeline/stages.py — the interface the rest of the pipeline calls

```
from pathlib import Path

ROOT   = Path(__file__).resolve().parent.parent
ART    = ROOT / "artifacts"

GROUPS_ALL     = ["alpha", "beta", "gamma"]
CHAIN_NEXT     = {"STAGE_A": "STAGE_B", "STAGE_B": None}   # what follows what
ANALYSIS_ONLY  = {"STAGE_B"}                # stages with no parallel work at all
NOTEBOOK       = {"STAGE_A": "notebooks/stage_a.ipynb",
                  "STAGE_B": "notebooks/stage_b.ipynb"}
LEDGER         = {"STAGE_A": ART / "tables" / "stage_a.csv",
                  "STAGE_B": ART / "tables" / "stage_b.csv"}
# The columns that uniquely identify one unit of work. `pending()` uses these to decide
# what has already been done, so they must be exactly the fields that vary across the
# sweep -- no more (or resumption misses work) and no fewer (or it redoes work).
KEYS           = {"STAGE_A": ["group", "arm", "seed"]}


def run_list(stage: str, groups: list[str]) -> list[dict]:
    """Every unit of work this stage needs, as plain dicts. The full sweep, not the
    remainder -- `pending()` subtracts what is done."""
    if stage == "STAGE_A":
        return [{"group": g, "arm": a, "seed": s}
                for g in groups for a in ("control", "treatment") for s in range(10)]
    return []


def execute(task: dict) -> dict:
    """Do ONE unit of work and return one flat, self-describing row.

    This is the only place the pipeline touches your actual project. It should:
      - be deterministic given the task dict (seed everything from it),
      - write its own heavy outputs (checkpoints, per-step metrics) under
        artifacts/runs/<key>/ itself,
      - return a SMALL dict of scalars -- the row the ledger and the notebook read.
    """
    from core.train import train_one            # your code, untouched by any of this
    result = train_one(**task)
    return {**task, **result}


def stage_groups(stage: str, groups: list[str]) -> list[str]:
    """Narrow a stage to a subset if it depends on an earlier stage's finding. Raise --
    do not silently return everything -- if the prerequisite is missing."""
    return groups
```

Two supporting functions belong here too and are almost identical in every project:

pipeline/stages.py — merge and resume

```
import json, pandas as pd

def merge_chunks(stage: str) -> None:
    """Fold every artifacts/chunks/<STAGE>_chunk_*.jsonl into the stage ledger, by NAME."""
    rows = []
    for p in sorted((ART / "chunks").glob(f"{stage}_chunk_*.jsonl")):
        for line in p.read_text().splitlines():
            if line.strip():
                rows.append(json.loads(line))
    if not rows:
        return
    led = LEDGER[stage]
    new = pd.DataFrame(rows)
    if led.exists():
        new = pd.concat([pd.read_csv(led), new], ignore_index=True)
    # Aligning by column NAME, and de-duplicating on the keys, is what makes a re-run of a
    # unit of work harmless rather than a duplicate row.
    new = new.drop_duplicates(subset=KEYS[stage], keep="last")
    led.parent.mkdir(parents=True, exist_ok=True)
    new.to_csv(led, index=False)


def pending(stage: str, wanted: list[dict]) -> list[dict]:
    """The subset of `wanted` with no row in the ledger. This function IS the resume
    logic; there is no state anywhere else."""
    led = LEDGER[stage]
    if not led.exists():
        return list(wanted)
    df = pd.read_csv(led)
    keys = KEYS[stage]
    have = {tuple(str(r[k]) for k in keys) for _, r in df.iterrows()}
    return [t for t in wanted if tuple(str(t[k]) for k in keys) not in have]
```

7.2 `manifest.py`

Builds the agreed list and writes it to disk. One decision inside it is not obvious and saves a lot of wall-clock time: **interleave the work across groups before slicing.**

The natural run list is grouped — all of group A, then all of group B. A contiguous split then hands every twenty-minute item to one task and every three-minute item to another, so one task runs for hours while another finishes in minutes and its node sits idle. Round-robin across groups first, and the chunks can stay simple contiguous slices while still carrying similar loads.

`pipeline/manifest.py`

```
#!/usr/bin/env python3
"""Build artifacts/manifests/<STAGE>.json -- the agreed run list for one stage."""
from __future__ import annotations
import argparse, json, sys
from datetime import datetime, timezone
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))
from pipeline.stages import (ANALYSIS_ONLY, ART, GROUPS_ALL,
                             merge_chunks, pending, run_list)


def interleave(tasks, groups):
    """Round-robin across groups so every chunk gets a comparable workload.

    Terminates on the longest per-group list. Write this as a bounded `for i in
    range(max_len)` loop, not `while any(...)`: the obvious while-loop version is an
    infinite loop the moment it forgets to drain, and the symptom is a manifest job that
    simply hangs until the walltime kills it.
    """
    by = {g: [t for t in tasks if t["group"] == g] for g in groups}
    leftover = [t for t in tasks if t["group"] not in by]   # would vanish silently otherwise
    out = []
    for i in range(max((len(v) for v in by.values()), default=0)):
        for g in groups:
            if i < len(by[g]):
                out.append(by[g][i])
    assert len(out) + len(leftover) == len(tasks), "interleave dropped work"
    return out + leftover


def manifest_path(stage: str) -> Path:
    return ART / "manifests" / f"{stage}.json"


def main() -> int:
    ap = argparse.ArgumentParser()
    ap.add_argument("--stage", required=True)
    ap.add_argument("--groups", default=",".join(GROUPS_ALL))
    ap.add_argument("--n_chunks", type=int, default=16)
    a = ap.parse_args()

    groups = [g.strip() for g in a.groups.split(",") if g.strip()]
    if a.stage in ANALYSIS_ONLY:
        tasks = []
    else:
        merge_chunks(a.stage)                       # fold in anything a previous pass did
        tasks = interleave(pending(a.stage, run_list(a.stage, groups)), groups)

    p = manifest_path(a.stage)
    p.parent.mkdir(parents=True, exist_ok=True)
    p.write_text(json.dumps({
        "stage": a.stage, "groups": groups, "n_chunks": a.n_chunks,
        "built_at": datetime.now(timezone.utc).isoformat(timespec="seconds"),
        "n_tasks": len(tasks), "tasks": tasks}, indent=1))

    # This line is the guard's payoff: it prints the number you compare against what you
    # meant to submit. Read it before you walk away from the terminal.
    print(f"[{a.stage}] {len(groups)} groups: {', '.join(groups)}")
    print(f"[{a.stage}] {len(tasks)} units pending -> {p.relative_to(ROOT)}")
    for g in groups:
        print(f"    {g:14s} {sum(1 for t in tasks if t['group'] == g):4d}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

7.3 `chunk.py` — one array task

Three rules are encoded here, each of which was learned the expensive way.

Write JSON Lines, not CSV. Rows from different groups do not all carry the same columns. Appending to a shared CSV with `header=False` writes values *positionally* under whatever header the first row created, so a row with a missing column shifts every later field one column to the left. Nothing errors; it surfaces weeks later as a string sitting in a numeric column. A JSON object per line carries its own field names and cannot be misread.

One file per task, never a shared one. Sixteen processes appending to a single file is how you get interleaved half-lines.

Never fail the task for one bad unit. Log it, keep going. And exit 0 regardless: unfinished work is normal and is the analyse job's business; a non-zero exit here only makes `sacct` look alarming without changing what happens next.

`pipeline/chunk.py`

```
#!/usr/bin/env python3
"""Process one chunk of a stage manifest. One invocation per SLURM array task."""
from __future__ import annotations
import argparse, json, sys, time, traceback
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))
from pipeline.manifest import manifest_path
from pipeline.stages import ART, execute


def contiguous_slice(items, chunk_id: int, n_chunks: int):
    """Even split, remainder spread over the first chunks. Every task computes the same
    sizes from the same list, so the slices partition it exactly."""
    base, rem = divmod(len(items), n_chunks)
    sizes = [base + (1 if i < rem else 0) for i in range(n_chunks)]
    start = sum(sizes[:chunk_id])
    return items[start:start + sizes[chunk_id]]


def main() -> int:
    ap = argparse.ArgumentParser()
    ap.add_argument("--stage", required=True)
    ap.add_argument("--chunk_id", type=int, required=True)
    ap.add_argument("--n_chunks", type=int, required=True)
    ap.add_argument("--minutes", type=float, default=280.0,
                    help="stop STARTING new work after this long (walltime minus margin)")
    ap.add_argument("--margin_min", type=float, default=25.0,
                    help="do not start a unit unless this much budget remains; size this "
                         "from the SLOWEST unit in the stage, not the average")
    a = ap.parse_args()

    mp = manifest_path(a.stage)
    if not mp.exists():
        print(f"no manifest at {mp} -- run slurm/01_manifest.sbatch first", file=sys.stderr)
        return 1
    mine = contiguous_slice(json.loads(mp.read_text())["tasks"], a.chunk_id, a.n_chunks)

    out = ART / "chunks" / f"{a.stage}_chunk_{a.chunk_id:03d}.jsonl"
    out.parent.mkdir(parents=True, exist_ok=True)
    print(f"[{a.stage} chunk {a.chunk_id}/{a.n_chunks}] {len(mine)} units mine", flush=True)
    if not mine:
        out.touch(); return 0

    t0 = time.time(); done = failed = skipped = 0
    with open(out, "a") as fh:                       # append: a requeued task adds to it
        for k, task in enumerate(mine, 1):
            left = a.minutes - (time.time() - t0) / 60.0
            if left < a.margin_min:
                skipped = len(mine) - k + 1
                print(f"[{a.stage} chunk {a.chunk_id}] {left:.1f} min left, {skipped} not "
                      f"started -- the analyse job will requeue them", flush=True)
                break
            print(f"\n[{a.stage} chunk {a.chunk_id}] ({k}/{len(mine)}) {task} "
                  f"({left:.0f} min left)", flush=True)
            try:
                fh.write(json.dumps(execute(task), default=str) + "\n")
                fh.flush()          # a task killed later must not lose what it finished
                done += 1
            except Exception:
                failed += 1
                print(f"*** FAILED {task} ***", flush=True)
                traceback.print_exc()
                # Keep going. One bad unit must not cost the other work in this slot; its
                # row is simply absent and the next pass picks it up.

    print(f"\n[{a.stage} chunk {a.chunk_id}] done={done} failed={failed} skipped={skipped} "
          f"in {(time.time()-t0)/60:.1f} min", flush=True)
```

```
    return 0                          # always 0: see the module docstring


if __name__ == "__main__":
    raise SystemExit(main())
```

7.4 `analyse.py` — requeue, or run the notebook and chain on

The two limits at the top of this file are what stop a requeue loop from running away. `MAX_PASSES` is a coarse backstop. `STALL_LIMIT` is the one that actually saves you: **a pass that completed nothing will not complete anything next time either.** The requeue loop exists for walltime cuts and dead nodes, where each pass finishes *some* work and the remainder shrinks. It is the wrong response to a fault that hits every unit identically — a missing dependency, an unreachable service, a config that raises for every group. Two consecutive zero-progress passes is enough to tell "everything is broken" from "the queue was slow".

pipeline/analyse.py

```
#!/usr/bin/env python3
"""Merge the chunks, then either requeue the stage or analyse it and chain onward."""
from __future__ import annotations
import argparse, json, os, subprocess, sys, traceback
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT)); os.chdir(ROOT)
from pipeline.stages import (ANALYSIS_ONLY, ART, CHAIN_NEXT, GROUPS_ALL, NOTEBOOK,
                             merge_chunks, pending, run_list, stage_groups)

MAX_PASSES = 40
STALL_LIMIT = 2          # consecutive zero-progress passes tolerated


def run_notebook(stage: str, groups: str) -> None:
    """Execute the stage's notebook code cells in order, in this process. Section 8."""
    os.environ["PROJ_GROUPS"] = groups        # how the notebook learns its scope
    import matplotlib; matplotlib.use("Agg")  # no display on a compute node
    nb_path = ROOT / NOTEBOOK[stage]
    nb = json.loads(nb_path.read_text())
    g = {"__name__": "__main__"}
    n = 0
    for i, c in enumerate(nb["cells"]):
        if c["cell_type"] != "code":
            continue
        src = "".join(c["source"]).replace("SMOKE = True", "SMOKE = False")
        n += 1
        print(f"\n----- {nb_path.name} cell {i} -----", flush=True)
        exec(compile(src, f"{nb_path.name}#cell{i}", "exec"), g)
    print(f"\n[{stage}] notebook complete: {n} code cells", flush=True)


def submit_stage(stage, groups, n_chunks, part_end, pass_no) -> None:
    """manifest -> array -> analyse, exactly as slurm/_submit_lib.sh does it.

    THE VARIABLES GO IN THE CHILD ENVIRONMENT, NOT IN sbatch's --export. See section 9.1:
    --export is a comma-separated list with no escaping, so a value containing commas is
    read as several items and silently truncated.
    """
    child = dict(os.environ)
    child.update(STAGE=stage, GROUPS=groups, N_CHUNKS=str(n_chunks),
                 N_GROUPS=str(len([g for g in groups.split(",") if g.strip()])),
                 PART_END=part_end or "", PASS_NO=str(pass_no))

    def sb(*args):
        r = subprocess.run(["sbatch", "--parsable", "--export=ALL", *args],
                           capture_output=True, text=True, check=True, env=child)
        return r.stdout.strip().split(";")[0]

    if stage in ANALYSIS_ONLY:
        print(f"  submitted analyse {stage} -> {sb('slurm/03_analyse.sbatch')}", flush=True)
        return
    mj = sb("slurm/01_manifest.sbatch")
    rj = sb(f"--dependency=afterok:{mj}", f"--array=0-{n_chunks-1}%16",
            "slurm/02_run_chunks.sbatch")
    aj = sb(f"--dependency=afterany:{rj}", "slurm/03_analyse.sbatch")
    print(f"  submitted {stage}: manifest={mj} runs={rj} analyse={aj}", flush=True)


def main() -> int:
    ap = argparse.ArgumentParser()
    ap.add_argument("--stage", required=True)
    ap.add_argument("--groups", default=",".join(GROUPS_ALL))
    ap.add_argument("--n_chunks", type=int, default=16)
    ap.add_argument("--part_end", default="")
    ap.add_argument("--pass_no", type=int, default=0)
    a = ap.parse_args()
    glist = [g.strip() for g in a.groups.split(",") if g.strip()]
```

```
# ---- 1. anything left? ------------------------------------------------------------
if a.stage not in ANALYSIS_ONLY:
    merge_chunks(a.stage)
    left = pending(a.stage, run_list(a.stage, glist))
    print(f"[{a.stage}] {len(left)} units still pending after merge", flush=True)

    sp = ART / "status" / f".{a.stage}_pending"
    prev = json.loads(sp.read_text()) if sp.exists() else None
    stalls = (prev["stalls"] + 1) if (prev and prev.get("left") == len(left)) else 0
    sp.parent.mkdir(parents=True, exist_ok=True)
    sp.write_text(json.dumps({"left": len(left), "stalls": stalls}))

    if left and stalls >= STALL_LIMIT:
        print(f"[{a.stage}] STOPPING: {len(left)} units pending for {stalls+1} "
              f"consecutive passes with ZERO completed in between. That is a fault "
              f"affecting every unit, not a walltime cut -- requeueing cannot fix it.\n"
              f"  Read:  tail -40 logs/run_*_0.err\n"
              f"  Nothing is lost: finished work is in the ledger and a resubmission "
              f"picks up from there once the fault is fixed.", flush=True)
        return 1

    if left:
        nxt = a.pass_no + 1
        if nxt > MAX_PASSES:
            print(f"[{a.stage}] STOPPING after {nxt} passes with {len(left)} pending.",
                  flush=True)
            return 1
        print(f"[{a.stage}] requeueing (pass {nxt})", flush=True)
        submit_stage(a.stage, a.groups, a.n_chunks, a.part_end, nxt)
        return 0

(ART / "status" / f".{a.stage}_pending").unlink(missing_ok=True)

# ---- 2. the analysis --------------------------------------------------------------
print(f"[{a.stage}] complete -- running {NOTEBOOK[a.stage]}", flush=True)
try:
    nb_groups = ",".join(stage_groups(a.stage, glist))
    run_notebook(a.stage, nb_groups)
except Exception:
    traceback.print_exc()
    print(f"\n[{a.stage}] ANALYSIS FAILED. The results are safe in the ledger. Fix the "
          f"notebook, then rerun only this job:\n"
          f"    export STAGE={a.stage} GROUPS='{a.groups}' N_GROUPS={len(glist)} "
          f"N_CHUNKS={a.n_chunks} PART_END={a.part_end} PASS_NO={a.pass_no}\n"
          f"    sbatch --export=ALL slurm/03_analyse.sbatch", flush=True)
    return 1

# ---- 3. gates, then the next stage ------------------------------------------------
# A downstream stage built on a failed check wastes hours and produces a result nobody
# can use. Let the notebook write a small JSON verdict and read it here.
status = ART / "status" / f"{a.stage}.json"
if status.exists():
    rec = json.loads(status.read_text())
    bad = [k for k, v in rec["gates"].items() if not v["pass"]]
    print(f"\n[{a.stage}] gates: {len(rec['gates'])-len(bad)}/{len(rec['gates'])} pass"
          + (f"   FAILED: {bad}" if bad else ""), flush=True)
    if bad:
        print(f"[{a.stage}] NOT advancing the chain. Read {status}.", flush=True)
        return 0

nxt = CHAIN_NEXT.get(a.stage)
if not nxt or a.stage == a.part_end:
    print(f"[{a.stage}] this part is finished.", flush=True); return 0
print(f"[{a.stage}] -> submitting {nxt}", flush=True)
submit_stage(nxt, a.groups, a.n_chunks, a.part_end, 0)
return 0
```

```
if __name__ == "__main__":
    raise SystemExit(main())
```

8 Executing the notebook

The analyse job runs the notebook rather than a script that duplicates it. Four things have to be true for that to work, and they are all cheap.

8.1 The notebook must resume, not retrain

Structure every experiment notebook the same way: a cell that computes the run list, a cell that runs whatever is missing, a cell that loads the ledger, then analysis. Locally you run it with a smoke setting and it does two minutes of wiring check. On the cluster the array has already produced every row, so the "run whatever is missing" cell is a no-op and the notebook goes straight to the analysis.

```
# --- cell 1: scope and mode ------------------------------------------------
import os
SMOKE  = True                                  # patched to False by the analyse job
GROUPS = os.environ.get("PROJ_GROUPS", "alpha").split(",")
SEEDS  = 2 if SMOKE else 10

# --- cell 3: produce anything missing --------------------------------------
# On the cluster this loop finds nothing to do. Locally it does a couple of tiny runs.
for task in pending(STAGE, run_list(STAGE, GROUPS)):
    execute(task)
merge_chunks(STAGE)

# --- cell 4 onward: the actual analysis ------------------------------------
df = pd.read_csv(LEDGER[STAGE])
```

8.2 Parameterisation: environment variables, not edits

The notebook must analyse exactly the scope the array trained. Pass it through the environment (`PROJ_GROUPS` above) and read it with a sensible default, so the same file opens and runs locally without any cluster variables set. Never let the notebook default to "everything" on a cluster run — it will look for results that were deliberately never produced and fail deep inside a plotting call.

8.3 Executing it: two options

Approach	Use when	Cost
In-process exec (the `run_notebook` above)	The default. No extra dependency, the traceback points at a real cell number, the log *is* the output, and you can patch a flag in memory without touching the file on disk.	No executed `.ipynb` is produced — the log and the written figures are the record.
`papermill`	You want an executed notebook, with outputs inline, as the archived artifact — or you need per-cell timing.	One more dependency and a second copy of every figure. `papermill in.ipynb out.ipynb -p groups "alpha,beta"`, with a `parameters`-tagged first cell.

If you take the in-process route, the flag patch (`SMOKE = True` → `SMOKE = False`) is done on the cell source string, in memory. The file on disk keeps the smoke setting *on*, so opening it locally is still a two-minute check rather than an accidental full run. Match the literal exactly, and keep it on its own line with the spaces shown — a string replacement that silently matches nothing gives you a full-length cluster job that quietly did the smoke version.

8.4 Headless plotting, always

`MPLBACKEND=Agg` is set in the prelude and `matplotlib.use("Agg")` is called again in `run_notebook` before the first import. Compute nodes have no display; a notebook that tries to open one fails at the first `plt.figure()`, after the expensive part is already done.

9 Guards that are not optional

Five failure modes, each of which produces a wrong result or a large bill while looking like success. Each is guarded in the templates above; they are collected here because a guard is only obvious once you know what it guards against.

9.1 `sbatch --export` splits on commas, and there is no escaping

```
sbatch --export=ALL,STAGE=E0,GROUPS=alpha,beta,gamma,N_CHUNKS=16  job.sbatch
```

Slurm reads that as **six** items, not four:

```
ALL | STAGE=E0 | GROUPS=alpha | beta | gamma | N_CHUNKS=16
```

The commas inside your value are separators. `GROUPS` arrives as `"alpha"`, and the bare words `beta` and `gamma` are read as *names of variables to propagate from the submitting environment* — variables that do not exist, so they are dropped without a word. Nothing errors. The stage runs one group out of three and exits 0.

The guard, in three parts. (1) Export the variables in the submitting shell and pass `--export=ALL` alone. (2) Send `N_GROUPS` alongside as a single integer that cannot be mangled the same way. (3) Have every receiving job refuse to run if the two disagree — that check is in `_prelude.sh`, so it applies to all of them. And make the manifest job print the group count it actually saw, so the guard has a human-readable receipt.

9.2 Telemetry must never be able to fail an experiment

Experiment trackers start a sidecar process per run and talk to it over a socket. Launch sixteen array tasks in the same second on a cluster with an NFS home and they will race; when that race is lost the tracker raises, the run dies before its first step, and — this is the expensive part — because nothing was recorded the analyse job sees the same work pending and requeues. Left alone, that loop can burn a full experiment's budget on runs that never took a step.

The guard, in three places. Wrap every telemetry call — init, log *and* the end-of-run upload — so a failure logs and continues. Set the tracker to `disabled` or `offline` in the array tasks, which removes the race rather than tolerating it; the single analyse process still logs the stage summary. And rely on `STALL_LIMIT` as the backstop. The rule underneath all three: **a monitoring sidecar must never be able to fail the thing it is monitoring.**

9.3 A stalled stage is not a slow stage

The requeue loop is correct for walltime cuts and dead nodes, where each pass completes some work. It is wrong for a fault that hits every unit identically, and it will happily retry that fault forty times. Track the pending count between passes; stop after two consecutive passes that completed nothing, and print what to read. A genuinely stalled stage loses at most one extra cycle; a genuinely broken one stops within minutes.

9.4 Positional CSV appends corrupt data silently

Covered in 7.3 and repeated because it is the one on this list that produces a *wrong number in a paper* rather than a failed job. Chunks write JSON Lines; the merge step aligns by column name; the ledger is written whole, never appended to positionally.

9.5 Leave walltime margin, sized from the slowest unit

A task that starts a forty-minute unit with twenty minutes of walltime left loses both the unit and the slot. Stop starting new work at `walltime − margin`, and size the margin from the slowest unit in the stage, not the average. The skipped work is requeued and costs nothing; the killed unit costs a node-hour and produces nothing.

10 Bring-up checklist

In order, on a cluster you have not used before. Each step is cheap and each one rules out a class of failure that is much harder to diagnose later.

#	Do	Why / what to read
1	`sinfo -s` `scontrol show partition`	Partition names, per-partition walltime limits, node counts. You cannot write a correct `#SBATCH` header without these.
2	`sacctmgr show assoc user=$USER \` `  format=account,maxjobs,maxsubmit`	Your concurrent and submitted caps. These set `CONCURRENCY` and `MAX_SUBMIT` in `_submit_lib.sh`.
3	`sinfo -o "%20N %10c %10m %25f"`	Node families and their *features* — the strings you can pass to `--constraint` when a wheel needs newer CPU instructions.
4	Find the priority bands	Your site's docs. Many clusters give a large bonus below a walltime threshold; ask for the band, not the maximum.
5	Check the filesystem you are on	`df -h ~` and your site's quota command. Home is usually NFS and usually small; scratch is usually fast and purged. Checkpoints belong on scratch, the ledger on something backed up.
6	`sbatch slurm/00_create_venv.sbatch`	Then read the whole log. Every import must pass. Rebuild before investigating anything else if several native packages fail identically.
7	`sbatch slurm/00_verify.sbatch`	Repository modules import; one real training step completes.
8	Submit one stage with `N_CHUNKS=2`	The smallest submission that exercises the whole chain: manifest, an array with more than one task, merge, requeue, notebook.
9	`cat logs/manifest_<id>.out`	**Before walking away.** It prints the number of groups it saw. Section 9.1 is the reason this step exists.
10	Cancel it mid-array, then resubmit	The only way to know your resume path works is to interrupt it once on purpose, while the stage is small.

Day-to-day commands

Command	Use
`squeue -u $USER`	What is queued and running
`squeue -u $USER --start`	Estimated start times for pending jobs
`squeue -j <id> -o "%.18i %.9P %.8T %R"`	Why a job is still pending (the reason field)
`sacct -j <id> --format=JobID,State,Elapsed,MaxRSS,ExitCode`	What a finished job actually used — size the next request from this, not from a guess
`scancel <id>` / `scancel -u $USER`	Cancel one job / everything. Safe: the ledger keeps what finished
`scancel <arrayid>_[5-15]`	Cancel part of an array
`tail -f logs/run_<A>_0.out`	Follow one array task
`grep -l Traceback logs/run_*_*.err | head`	Which tasks raised
`scontrol show job <id>`	Everything Slurm knows about a job, including the resolved `--export` environment

11 Adapting this to a new project

The folder is deliberately boring so that porting it is a short list. In order:

1. **`_prelude.sh`** — set `PROJ_VENV`. If the project is not CPU-only, remove the `CUDA_VISIBLE_DEVICES=""` line and add `--gres=gpu:1` to the array's header.
2. **`_submit_lib.sh`** — set `MAX_SUBMIT`, `CONCURRENCY` and `default_groups` from checklist steps 1–2.
3. **`00_create_venv.sbatch`** — the framework install line, and one `check` line per top-level dependency.
4. **`02_run_chunks.sbatch`** — walltime, cpus, memory, and the `--minutes` / `--margin_min` pair, from the slowest unit of work.
5. **`pipeline/stages.py`** — the real work: `run_list`, `execute`, `KEYS`, `NOTEBOOK`, `CHAIN_NEXT`. This is the only file with any experiment in it.
6. **Each notebook** — add the scope-from-environment cell and the smoke flag, and make sure the "produce anything missing" cell is a no-op when the ledger is full.

Nothing else should need to change. If you find yourself editing an `.sbatch` file to express something about your experiment, that logic belongs in `stages.py` instead — that is the whole reason the boundary is drawn where it is.

> **THE ONE-PARAGRAPH VERSION**
>
> Nothing runs on the login node. One job writes the run list, an array of tasks each takes a contiguous slice of it and appends self-describing JSON rows, and one job merges those and either requeues what is left or executes the notebook. Resumption is a set difference against a ledger, so a walltime cut, a dead node and a cancelled submission all cost the same thing. Export variables in the submitting shell and pass `--export=ALL` alone. Telemetry can never be allowed to fail a run. And stop requeueing after two passes that completed nothing, because they will not complete anything on the third.